🧠 ODSA-AIM_and_METHOD

Aim

This project characterises Antarctic subglacial landscape features through statistical and spectral analysis of bedrock topography. Historical radio-echo sounding (RES) flight lines provide bedrock elevation measurements along discrete transects. The Reference Elevation Model of Antarctica (REMA) supplies contemporary surface elevations, from which ice thickness and local ice-flow direction are derived. Together, these datasets enable roughness classification and anisotropy analysis across multiple Antarctic regions.

This analysis comprises a **code base of 4 different scripts**

1. `bed_analysis.py`
2. `REMA_extractor.py`
3. `incidence_scatter.py`
4. `flow_plots.py`

ODSA_README

Method

1. Data Ingestion & Pre-processing (`bed_analysis.py`)

- **Configurable Ingestion:** The load_datasets() function uses a dictionary-based configuration (target_files) where each entry specifies a label and an optional subsetting function. It returns a list of dictionaries (`all_dfs`), each containing the region name and its DataFrame.
- **Trajectory Filtering:** The script specifically targets unique `trajectory_id` values and automatically discards any trajectory with fewer than **20 data points**
- **Coordinate Transformation:** Using `pyproj.Transformer` with `always_xy=True` , coordinates are converted from WGS84 to Antarctic Polar Stereographic (EPSG:3031). This allows for accurate cumulative along-track distance calculation in meters.
- **Segmentation and Filtering**: The function split_into_segments() identifies along-track gaps exceeding 2 km and splits each trajectory into independent segments. This prevents spectral analysis from smearing data across physical voids. Segments must satisfy both a minimum point count (50) and a minimum length (10 km) to qualify for analysis, ensuring sufficient spectral bandwidth for the Lomb-Scargle periodogram.

2. Spatial Domain Analysis (Per Segment)

Bedrock Detrending is applied at two scales. First, a single linear detrend is applied to the full segment to remove the large-scale slope; this is used for the spatial profile plots. Separately, within each **sliding window** (50 km, 80% overlap), a local linear detrend isolates the topographic residuals used for spectral analysis and RMS roughness calculation, as well as the morphometric statistics (local relief, RMS roughness, largest local structures). The plots display the full-segment detrend rather than the per-window detrending.
NOTE if a segment is shorter than 50 km, the code falls back to treating the entire segment as a single window (`window_size=segment_len_m`).

2.1 Bedrock Spectral Signal Processing

refs: [Jordan_2017, Bingham_2009, VanderPlas_2018]

Frequency domain analysis for non-periodic and unevenly spaced real-world data

a. **Window parameters**: A "rectangular" window (no tapering) is applied to each sliding segment, preserving signal energy and providing the finest frequency resolution. Although rectangular windows exhibit greater spectral leakage than tapered alternatives, this trade-off is acceptable given the power law fitting approach.
Window Size: 50 km. **Step Size:** 10 km.
Overlap: $\frac{50k-10k}{50k} = 0.8 =$ 80% between consecutive windows.

b. **Lomb-Scargle Periodogram:** This algorithm performs least-squares spectral analysis without requiring resampling or uniform spacing. Instead, it fits sinusoids directly to the data at their original along-track positions—well suited to radar data, which is rarely evenly spaced due to variations in aircraft ground speed. The script calculates a Lomb-Scargle periodogram over a frequency grid spanning from `1/window_size` to the Nyquist limit. The power-law fit is then restricted to a 250m–50km wavelength band to ensure consistency across regions.

c. **PSD averaging**: The per-window periodograms are averaged to produce a robust PSD. A two-pass power law fit is applied to this averaged PSD.

d. **Two-pass power-law fit**:
First Pass: The script fits an initial power law to the PSD and identifies significant peaks in the residuals.
Second Pass: The second-pass fit masks peaks with a buffer of ±5 frequency bins to isolate the background roughness spectrum, and the power law is refitted to the *cleaned* data.

e. **Whitened spectrum** is calculated by dividing the full PSD by the cleaned power-law fit.
This flattens the background roughness trend, making spectral peaks associated with larger bedforms more prominent.

f. **Peak Finding:** The script uses `scipy.signal.find_peaks` to identify peaks in the whitened spectrum that exceed the background trend by a factor of two or more. These peaks correspond to dominant topographic wavelengths within each flight line. Detected wavelengths are aggregated across all flight lines to assess whether a *characteristic* periodicity exists at the basin scale.
A minimum sampling floor is enforced when calculating the Nyquist limit: `max(dx_median, 15.0)`. This ensures a minimum spacing of 15m per window, preventing aliasing artifacts caused by GPS jitter or overlapping points at shorter wavelengths.

g. **Wavelength validation**: Only *confirmed* wavelengths are included in the final statistics. A wavelength is confirmed if $\lambda < L/2$, where L is the segment length. Wavelengths exceeding this threshold are classified as *candidates* and excluded, since fewer than two complete cycles fall within the observation window (making them indistinguishable from linear trends or red noise). The detectable range is explicitly constrained to 250m – 50km. Wavelengths longer than the observation window cannot be reliably resolved and introduce spectral leakage, causing trends to mimic periodic signals.

h. **The spectral slope** (β) is saved and displayed in the final results summary. This parameter characterises the scale-dependent roughness of the terrain:

$$\mathrm{PSD}(f) \propto f^{-\beta} \Leftrightarrow \mathrm{PSD}(f) \propto \lambda^{\beta}$$

Beta β	**Landscape Type**	**Interpretation**
< 1.5	**White Noise**	**Chaotic.** Can indicate data noise, OR **shattered crystalline bedrock** (e.g., deep glacial troughs) [VanderPlas_2018]
~2.0	**"Brownian" / Fractal**	**Hard Bed.** Typical of crystalline shields (granite/gneiss). The ice has to "stick and slip" over these jagged teeth. Friction is high. The curve is flatter. This means there is a lot more "power/energy/amplitude" at high frequencies (small wavelengths). [Jordan_2017]
> 2.5	**Smoothed / Draped**	**Soft Bed.** Likely a sedimentary basin (mud/till). The ice can slide rapidly over this deformable sludge. This is often where you find fast-flowing **Ice Streams**. The curve is steep. This means the "power" drops off very fast at high frequencies. [Bingham_2009]

Note

These classification boundaries are approximate and that β alone doesn't uniquely determine landscape class. ***Relief***, ***anisotropy***, *and* ***wavelength distribution*** *together give a more robust classification than any single metric.*

Hurst exponent ζ is a measure of surface self-affinity, is derived from the spectral slope as:

$$\zeta = \frac{\beta - 1}{2}$$

this metric is reported in the final results as `hurst_exponent` in the `bed_analysis` script.

2.2 Plotting PSD (`plot_spectra`)

This function generates a three-panel figure:

- `ax1` : Bedrock elevation profile (black line).
- `ax2` : Log-log plot of wavelength versus PSD, with the power-law fit overlaid.
- `ax3` : Whitened spectrum (residuals), used to assess whether detected wavelengths represent physical bedforms or noise.

Peak wavelengths are marked with coloured points: **blue** for the minimum detected wavelength and **red** for the maximum.
NOTE: The code limits spectral plots to the first 10 segments (`if plot_count < 10`).

2.3 Segment-to-Trajectory Aggregation

Because a single trajectory may contain multiple valid segments (after gap splitting, length filtering, and thickness validation), the script must combine per-segment results into a single trajectory-level record. The aggregation rules differ by metric type:

Scalar metrics that describe extremes are aggregated accordingly: `elevation_min` takes the minimum across segments, `elevation_max` the maximum, and `max_local_relief` retains the largest value found in any segment (with its corresponding location preserved). The `profile_length` and most other scalar statistics (RMS roughness, ice thickness) are averaged across segments.

Metrics needed for downstream regression — specifically β, ζ, `flow_orientation`, and `flow_incidence_deg` — are *collected* as lists rather than collapsed into a single value per trajectory. This preserves segment-level resolution for the $\beta = a + b\cos^2\theta + \varepsilon$ regression and avoids masking genuine within-trajectory variability in roughness anisotropy.

List-valued metrics (dominant wavelengths, confirmed/candidate wavelengths, window-level statistics) are flattened across segments. For example, if segment **A** found wavelengths [5 km, 12 km] and segment B **found** [8 km], the trajectory record stores [5, 12, 8] km.

String-valued metrics (flow orientation at the trajectory level, used only for display) take the mode across segments.

3. Other Statistical analysis

Surface elevation extraction, **flow vector computation**, and **ice thickness derivation** are handled by a companion module (`REMA_extractor.py`) which wraps the REMA mosaic access. The `analyse_bedrock()` function loop in `bed_analysis.py` iterates through this list and proceeds to process each region separately to analyze the topography.

3.1 Ice Thickness

The workflow `in bed_analysis` calculates ice thickness by integrating local radar-derived bedrock data with the **Reference Elevation Model of Antarctica (REMA)** surface mosaic using `REMA_extractor.py`.

Thickness Threshold: The script introduces a strict **validity check for ice thickness**. Segments are now automatically discarded if less than **20%** of the data has valid thickness measurements. The thickness calculation itself (including how surface elevations are derived from REMA and how H is validated) is detailed in **Section 3.1**. The threshold here acts as an additional segment-level filter applied before spectral analysis begins.

To ensure physical consistency, the script applies two primary filters:

- **NoData Handling:** REMA "nodata" values are converted to NaNs to prevent interpolation artifacts.
- **Physical Constraint:** Any calculated thickness values <0 (where the measured bed appears higher than the surface) are set to NaN to maintain dataset integrity.

3.2. Surface Elevation Extraction

Surface elevations are extracted from the REMA DEM at specific flight track coordinates using **Bilinear Interpolation**.

```
from scipy.ndimage import map_coordinates
elevations = map_coordinates(data, [rows, cols], order=1, mode='nearest')
```

The mode='nearest' parameter ensures edge cases do not return null values." (as a third bullet alongside Coordinate Mapping and Interpolation)

- **Coordinate Mapping:** Projected world coordinates (EPSG:3031) are mapped to image pixel indices using an inverse affine transform.
- **Interpolation:** A first-order spline interpolation (`order=1`) calculates the surface height at the exact point of the radar measurement, ensuring high spatial accuracy that matches JR's Fortran-based method.

3.3. Thickness Derivation

Ice thickness (H) is determined by the difference between the interpolated surface elevation (z_s) and the radar-measured bedrock altitude (z_b):

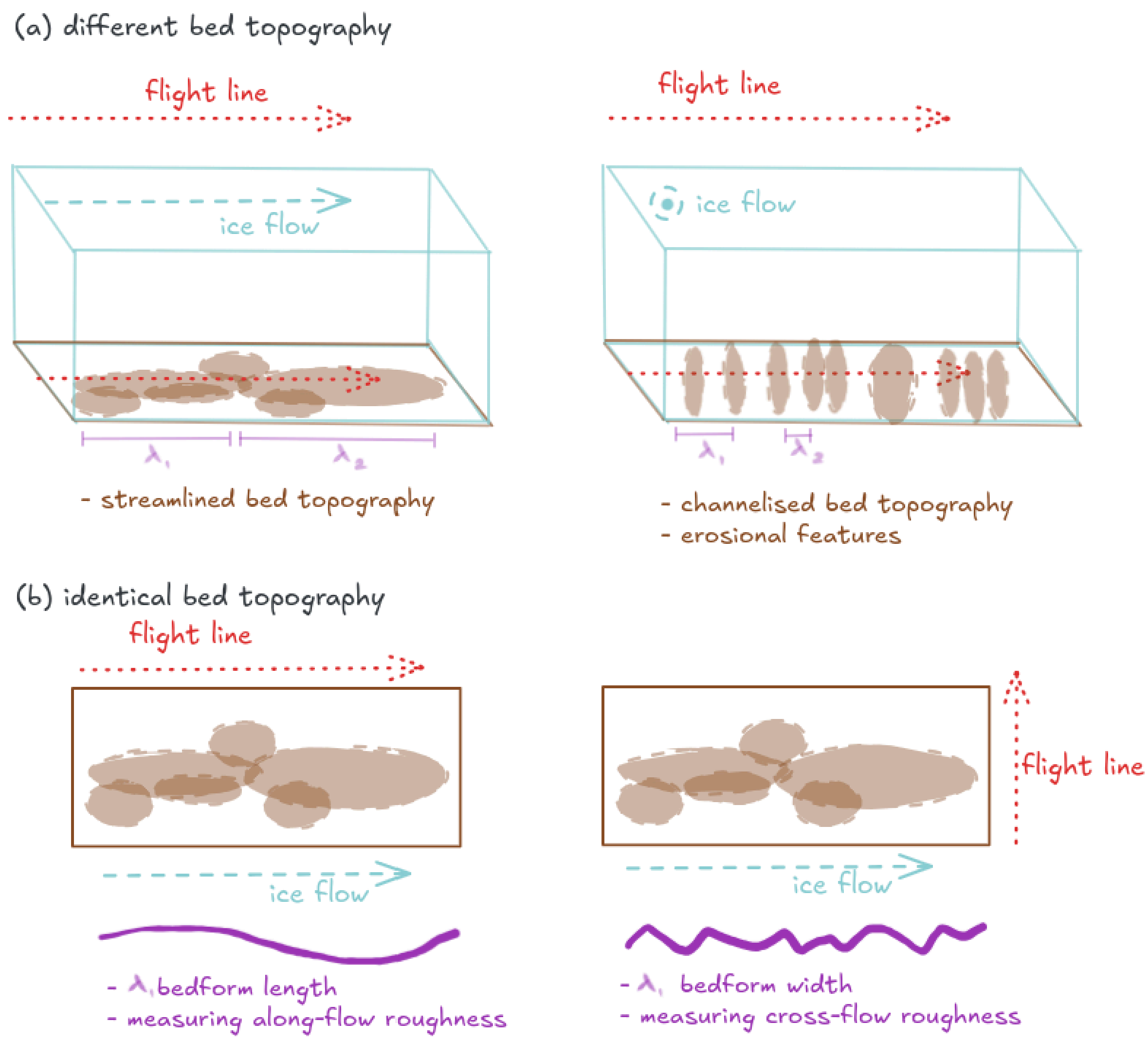
$$H = z_s - z_b$$

3.4. Determining "True" Flow Direction For Each Flight Line (Anisotropy)

ref [McCormack_2019]

The physical interpretation of detected wavelengths depends on local ice flow direction, because bedrock roughness is anisotropic—it varies with measurement orientation. As described in **Section 2.1**, the spectral slope β characterises the scale-dependent roughness of the terrain, and its value varies systematically with flight line orientation relative to ice flow:

- **Parallel to ice flow:** Flight lines sample along-flow roughness. Higher β values indicate reduced high-frequency power, consistent with streamlined, glacially smoothed terrain.
- **Perpendicular to ice flow:** Flight lines sample cross-flow roughness. Lower β values indicate greater high-frequency power, reflecting rougher, less modified terrain.



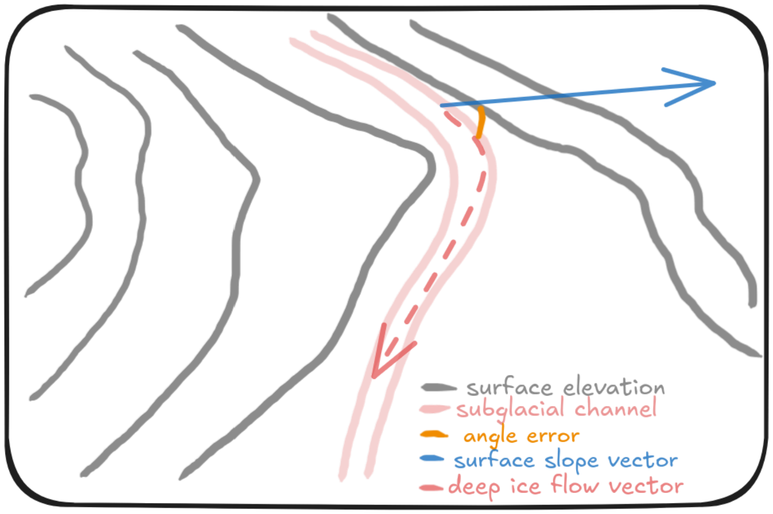
Schematic illustrating the anisotropic interpretation of detected wavelengths.

(a) Different beds, same flight line orientation: detected wavelengths (λ) differ due to genuine geological variation.
(b) Same bed, different flight line orientation: detected wavelengths differ because orthogonal transects sample different dimensions of the same features.

A detected wavelength may therefore represent either **bedform length** — when the flight line is parallel to ice flow —or **bedform spacing**— when the flight line is perpendicular to ice flow.

3.5. Local Surface Gradient

In deep, fast-flowing channels, the direction the **surface is tilting** (slope) is often **NOT** the same direction the **bottom ice is moving** (flow).



McCormack et al. (2019) addressed this mismatch by smoothing the surface DEM with a spatially varying triangular filter of width $8\text{–}10 \times H$ (eight to 10 times the ice thickness). This filtering removes local surface undulations that are decoupled from the bed, leaving only the broad regional slope that actually drives deep ice flow. Crucially, the smoothing is used **only** to derive flow orientation —it does not modify the bedrock elevations used in the spectral power-law fit. The smoothed slopes showed strong agreement with observed satellite velocity fields (MEaSUREs v2).

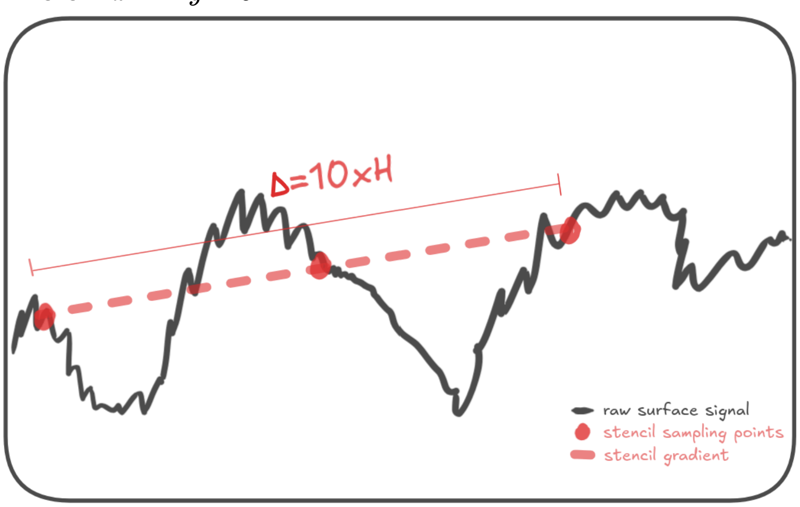
Here, the regional **ice flow vector** is estimated from the REMA mosaic (100 m resolution) using the surface elevation gradient:

$$v_{flow} \propto -\left(\frac{dS}{dx}, \frac{dS}{dy}\right)$$

where S is ice surface elevation. A **central finite difference stencil** captures the large-scale flow direction by sampling the elevation at four neighbouring points:

$$\frac{\partial S}{\partial x} \approx \frac{z_{i+1,j} - z_{i-1,j}}{2\Delta x}, \\ \frac{\partial S}{\partial y} \approx \frac{z_{i,j+1} - z_{i,j-1}}{2\Delta y},$$

Where $\Delta x = \Delta y = 5 \times H$.



This stencil acts as a **low-pass filter**, ignoring roughness features smaller than the stencil width. It achieves a similar goal to McCormack's triangular smoothing — suppressing local noise to find regional flow — but is faster to calculate. Gradients are sampled at flight line coordinates, allowing the **incidence angle** between

each flight path and the local ice flow direction to be calculated.
Note: Flat regions are handled by imposing a unit magnitude when the gradient is zero-valued to avoid division-by-zero errors when normalizing the flow vector.

4. Flight Line Classification: Roughness Anisotropy and Regression

The `bed_analysis.py` script categorises flight lines by orientation relative to flow using the dot product to calculate angles between the flight line and flow direction, ignoring upstream/downstream sense. Orientation thresholds are: **Parallel** < 30°, **Oblique** 30–60°, and **Perpendicular** > 60°.

`incidence_scatter.py` : This script performs a continuous regression analysis to quantify how topographic roughness (β) varies as a function of the measurement orientation relative to ice flow (θ). This replaces traditional categorical binning with a physics-based anisotropy model.

4.1. Multi-Scale Data Integration

The analysis is conducted at two distinct scales, utilizing data exported from the primary processing pipeline:

- **Window-Level Analysis (`{region}_window_stats.csv`):** Uses 50 km sliding windows with 80% overlap. A single-pass `np.polyfit` is applied to the log-log PSD of each window without peak masking. This produces a high-density dataset for visualizing continuous variance and calculating linear correlation statistics (R^2, p-value).
- **Segment-Level Analysis (`{region}_segment_stats.csv`):** Uses the more robust, two-pass iterative β values derived from the full-segment averaged PSD. This scale is used for the formal anisotropy curve-fitting because it minimizes the impact of local noise and transient spectral peaks.

4.2. Anisotropy Modeling ($\cos^2\theta$ fit)

The script uses `scipy.optimize.curve_fit` to fit a periodic anisotropy model to the segment-level data. The model assumes that roughness is a combination of along-flow smoothing and cross-flow channelization:

$$\beta(\theta) = \beta_\perp + (\beta_\parallel - \beta_\perp)\cos^2\theta$$

- θ **(Incidence Angle):** The angle between the flight path and the regional ice flow vector.
- $\beta_\parallel$ **(Parallel):** The solved spectral slope for flight lines perfectly aligned with flow; higher values typically indicate streamlined, glacially smoothed terrain.
- $\beta_\perp$ **(Perpendicular):** The solved spectral slope for flight lines cross-cutting flow; lower values typically reflect the high-frequency power of channelized or erosional features.
- $\Delta\beta$**:** Calculated as $\beta_\parallel - \beta_\perp$ to represent the total magnitude of roughness anisotropy in the region.

4.3. Optimization Logic

To ensure convergence, the script implements the following automated steps:

- **Initial Guessing (p_0):** Seeds the optimizer by calculating the mean β for segments in the Parallel ($< 30°$) and Perpendicular ($> 60°$) bins.
- **Error Estimation:** Returns the standard deviation errors for both $\beta_\parallel$ and $\beta_\perp$ using the square root of the covariance matrix diagonal.

- **Statistical Evaluation:** Calculates the Residual Sum of Squares (RSS) and Total Sum of Squares (TSS) to provide a final R^2 for the non-linear fit.

---

5. Spatial Context & Visualization

`flow-plots.py` : This script generates map-view plots that provide spatial context for each analysed region. Each plot overlays flight-line tracks on a REMA hillshade background, with segments coloured by orientation class (parallel, oblique, or perpendicular). Blue vectors indicate ice-flow direction at the surface, and each segment is labelled with its average flow angle.

References:

@Article{Jordan_2017,
author = {T M Jordan and M A Cooper and D M Schroeder and C N Williams and J D Paden and M J Siegert and J L Bamber},
title = {{Self-affine subglacial roughness: consequences for radar scattering and basal water discrimination in northern Greenland}},
journal = {The Cryosphere},
volume = {11},
year = {2017},
number = {3},
pages = {1247--1264},
doi = {10.5194/tc-11-1247-2017}
}
@article{Bingham_2009,
author = {R G Bingham and M J Siegert},
title = {{Quantifying subglacial bed roughness in Antarctica: implications for ice-sheet dynamics and history}},
journal = {Quaternary Science Reviews},
volume = {28},
number = {3},
pages = {223-236},
year = {2009},
note = {Special Theme: Modern Analogues in Quaternary Palaeoglaciological Reconstruction (pp. 181-260)},
issn = {0277-3791},
doi = {https://doi.org/10.1016/j.quascirev.2008.10.014},
}
@article{VanderPlas_2018,
author = {J T VanderPlas},
title = {{Understanding the Lomb–Scargle Periodogram}},
year = {2018},
month = {may},
publisher = {The American Astronomical Society},

volume = {236},
number = {1},
pages = {16},
journal = {The Astrophysical Journal Supplement Series},
doi = {10.3847/1538-4365/aab766},
}
@article{McCormack_2019,
author = {F S McCormack and J L Roberts and L M Jong and D A Young and L H Beem},
title = {{A note on digital elevation model smoothing and driving stresses}},
journal = {Polar Research},
volume = {38},
pages = {3498},
year = {2019},
doi = {10.33265/polar.v38.3498},
}